The Journal of Open Source Software

TensorTrainNumerics.jl: A Julia package for tensor train numerical methods

Martin Mikkelsen [1]

1 University of Copenhagen, Department of Computer Science, Denmark

DOI: 10.xxxxxx/draft

Software

- Review
- Repository
- Archive

Editor: Owen Melia

Reviewers:

- @emapuljak
- @zazabap
- @ion-g-ion

Submitted: 15 October 2026
Published: unpublished

License
Authors of papers retain copyright and release the work under a Creative Commons Attribution 4.0 International License (CC BY 4.0).

Summary

Many problems in scientific computing involve arrays too large to store directly, such as functions sampled on very fine grids, discretizations of partial differential equations in several dimensions, or functions of many variables. When these arrays have low-rank structure, they can be stored and manipulated in compressed form, so that such problems can be solved without ever forming the full array. `TensorTrainNumerics.jl` is a Julia package for this kind of computation. It provides compressed vectors and operators with arithmetic written in mathematical notation, explicit compressed representations of finite-difference operators (Kazeev & Khoromskij, 2012) and of common functions, construction of compressed arrays from function evaluations by cross interpolation (Dolgov & Savostyanov, 2020; Oseledets & Tyrtyshnikov, 2010), and interchangeable solvers for linear systems, eigenvalue problems and time evolution.

The compression used is the tensor train (TT) format, which writes a d-dimensional array as a chain of small three-index tensors, so that the storage and the cost of basic linear algebra grow linearly in d rather than exponentially, provided the ranks of the chain stay bounded (Oseledets, 2011). The package puts particular emphasis on the quantics tensor train (QTT) format, which applies the same decomposition to the binary digits of a grid index: a function on a grid with 2^N points can then be stored and manipulated at a cost linear in N, again provided the ranks stay bounded (Khoromskij, 2011).

Statement of need

Tensor train methods for numerical analysis are developed mainly in the applied mathematics literature, while the general-purpose tensor network packages in Julia, such as `ITensors.jl` and `MPSKit.jl`, focus primarily on quantum many-body physics, and their interfaces are organized around lattice models and matrix product states. `TensorTrainNumerics.jl` is aimed at researchers and students in numerical analysis and scientific computing who want to solve partial differential equations on very fine grids or in many dimensions, eigenvalue problems, and high-dimensional interpolation and integration problems in the TT and QTT formats. The package started as a tool for learning tensor train numerics, and it is organized so that a new algorithm can be added next to the existing ones and compared with them on the same problem. The repository includes runnable examples for each of these problem classes, and the package has been used in a publication (Arenstein et al., 2026) and a preprint (Mikkelsen & Kastoryano, 2026).

State of the field

The packages closest in scope are the MATLAB TT-Toolbox (Oseledets & TT-Toolbox contributors, 2014) and its Python counterpart `ttpy` (Oseledets & ttpy contributors, 2017),

DRAFT

which provide TT arithmetic, cross approximation, linear and eigenvalue solvers, and QTT representations of Laplace operators, and the Python package `scikit_tt` (Gelß & Scikit-TT contributors, 2022), which provides linear, eigenvalue and time-evolution solvers in the TT format with a focus on high-dimensional dynamical systems. In Julia, `ITensors.jl` (Fishman et al., 2022) and `MPSKit.jl` (Devos et al., 2026), the latter built on `TensorKit.jl` (Devos & Haegeman, 2025), are designed primarily for quantum many-body physics, `MPSTime.jl` (Moore et al., 2025) applies matrix product states to time-series machine learning, and `TensorCrossInterpolation.jl` (Núñez Fernández et al., 2025) constructs tensor trains by cross interpolation.

Several of these packages cover parts of tensor train numerical analysis: TT-Toolbox and `scikit_tt` provide solvers in the TT format, `TensorCrossInterpolation.jl` builds tensor trains from function evaluations, and the physics-oriented Julia libraries provide DMRG-type eigenvalue solvers. `TensorTrainNumerics.jl` differs in combining three things in one Julia package: a library of QTT operators on multidimensional grids, including finite-difference, shift, Fourier (Chen & Lindsey, 2024) and prolongation operators (Lubasch et al., 2018); solvers for linear systems, eigenvalue problems and time evolution that can be exchanged for one another, including inside implicit time integrators; and reverse-mode automatic differentiation through tensor train operations. The repository's Gross–Pitaevskii sensitivity example shows what this combination allows: it assembles QTT operators on a sequence of grids, computes a ground state with a multigrid method and a penalty-based ALS solver, differentiates the ground-state energy with respect to the trap frequency and the interaction strength with `Zygote.jl`, and uses these derivatives in Newton's method to find the interaction strength that gives a prescribed energy, all in one program.

Software design

A tensor train is stored as a plain vector of Julia arrays: the core G_k of a `TTVector` is an array of size (n_k, r_{k-1}, r_k), and the cores of a `TTOperator` carry an additional column index, matching the formulas in the literature (Khoromskij, 2018). This keeps algorithms close to their written description, lets contractions be expressed with `TensorOperations.jl`, and supports real and complex element types with the same code. Arithmetic extends Julia's own functions, such as `+`, `*`, `kron` (also written `⊗`) and `dot`, such that the two-dimensional Laplacian is written as `Δ ⊗ I + I ⊗ Δ`. The cost of this choice is that indices are identified by position rather than by name, as in `ITensors.jl`.

The same QTT cores represent different functions depending on how the bits of several spatial dimensions are arranged, either one dimension after another or interleaved. The types `QTTVector` and `QTTOperator` therefore record the number of spatial dimensions, the number of bits per dimension and the bit ordering. This lets conversions to grid arrays and the assembly of multidimensional operators use the correct ordering, and lets mismatched operands be detected with `check_compat` instead of producing a silently wrong result. To avoid writing every operation twice, operations are defined once on abstract supertypes, and the QTT information is attached to the result afterwards. QTT operators and functions are constructed directly in low-rank form rather than by compressing dense arrays, so their cost does not depend on the full grid size 2^N. These include finite-difference Laplacians (Kazeev & Khoromskij, 2012), the discrete Fourier transform (Chen & Lindsey, 2024), prolongation operators between grid levels (Lubasch et al., 2018), and functions such as $\sin(\lambda\pi x)$ and Chebyshev polynomials.

Solvers are selected by algorithm objects, such as `AMEn()` (Dolgov & Savostyanov, 2014) or `MALS()` (Holtz et al., 2012), which carry the method's parameters and are passed to `linear_solve`, `eigen_solve`, `time_evolve` or `tt_cross`. Implicit time integrators take a linear solver algorithm as a parameter, so any linear solver can be used inside any implicit scheme, and a new method is added by defining a new algorithm type and a method for it, after which it can be compared with the existing solvers on the same problem. The package also integrates with other Julia packages. The TT types implement the `VectorInterface.jl` interface, so

DRAFT

they can be used directly with Krylov (Haegeman, 2026) and optimization packages that accept generic vector types. Three integrations are optional and are package extensions that load only when the corresponding packages are loaded: an extension using `ChainRulesCore.jl` (White et al., 2026) defines reverse-mode differentiation rules for tensor train operations, so that `Zygote.jl` can differentiate through them; an extension for `Manopt.jl` (Bergmann, 2022) allows optimization directly over tensor trains; and an extension for `TensorCrossInterpolation.jl` (Núñez Fernández et al., 2025) converts its tensor trains to the package's format, including those produced by the interpolative QTT constructions of `InterpolativeQTT.jl` (Lindsey, 2023).

Research impact statement

`TensorTrainNumerics.jl` started as a tool to support the author's own research and to learn tensor train numerics, and it has since been used in two research projects. The numerical experiments on quantics tensor train solvers for differential equations with data integration in Arenstein et al. (2026) were carried out with the package, and the tensor train multidimensional inverse Laplace transform of Mikkelsen & Kastoryano (2026) is built on its tensor cross interpolation. The package has also received contributions from two external contributors, including an example that solves the Stein–Poisson equation for ϕ^4 lattice field theory following the approach of Butti et al. (2026).

The package is registered in the Julia General registry and archived on Zenodo, and it has had 17 tagged releases since its first commit in July 2024. The repository contains about 40 runnable example scripts, covering the Allen–Cahn equation, Fokker–Planck equation, Schrödinger and Gross–Pitaevskii equations, ground states of spin chains, and tensor cross interpolation and integration of high-dimensional functions. Together with the explicit operator constructions, these examples are intended to lower the barrier to entry for students and researchers who are new to tensor train numerics.

Usage

As an example, we solve Laplace's equation $\Delta u = 0$ on the unit square with $u(x, 0) = \sin(\pi x)$ and $u = 0$ on the other three sides, whose exact solution is $u(x, y) = \sin(\pi x)\sinh(\pi(1 - y))/\sinh(\pi)$. We use the standard five-point finite-difference scheme on $2^d \times 2^d$ interior grid points with spacing $h = 1/(2^d + 1)$. The one-dimensional second-difference matrix is built directly in QTT format with `toeplitz_to_qtto`, and the two-dimensional Laplacian is assembled from it with Kronecker products:

```
using TensorTrainNumerics

d = 8; h = 1 / (2^d + 1)
Δ1d = toeplitz_to_qtto(-2.0, 1.0, 1.0, d)
A = (1 / h^2) * (Δ1d ⊗ id_tto(d) + id_tto(d) ⊗ Δ1d)
```

The boundary values on $y = 0$ enter the right-hand side as the product of $\sin(\pi x)$ on the grid and the first unit vector in y. We start from a random tensor train with the same ranks as the right-hand side:

```
b = -(1 / h^2) * qtt_sin(d; a = h, b = 1 - h) ⊗ qtt_basis_vector(d, 1)
x0 = rand_tt(b.dims, b.ranks)
```

The system is solved with `AMEn`, replacing `alg` with `MALS()` or `DMRG()` selects a different solver without other changes:

```
alg = AMEn(; tol = 1e-10, max_sweeps = 10)
x = linear_solve(A, b, x0, alg)
```

DRAFT

```
u = qttv_to_array(QTTVector(x, 2, d, :serial))
```

The last line returns the solution as a $2^d \times 2^d$ array. The computed solution has maximum rank 2 and is stored in 112 numbers instead of the 65,536 grid values. Its relative error with respect to the exact solution is 7.6×10^{-6}.

AI usage disclosure

No generative AI tools were used in the initial development of this software. After release `v1.1.1` agentic coding with Claude Code (Anthropic) has been used to improve the code quality, write docstrings and extend test coverage. Generative AI was used to edit parts of this manuscript. All text produced with AI assistance was reviewed, revised, and verified by the author before inclusion in the code or paper.

Acknowledgements

The author thanks Shuhei Ohno and Louis Spatscheck for their contributions.

References

Arenstein, L., Mikkelsen, M., & Kastoryano, M. (2026). Fast and flexible quantum-inspired differential equation solvers with data integration. *Mathematics*, *14*(12). https://doi.org/10.3390/math14122069

Bergmann, R. (2022). Manopt.jl: Optimization on manifolds in Julia. *Journal of Open Source Software*, *7*(70), 3866. https://doi.org/10.21105/joss.03866

Butti, P., Catumba, G., Nada, A., & Spatscheck, L. (2026). A variational framework for variance reduction in lattice field theory. *arXiv Preprint arXiv:2609.28142*.

Chen, J., & Lindsey, M. (2024). *Direct interpolative construction of the discrete Fourier transform as a matrix product operator*. https://arxiv.org/abs/2404.03182

Devos, L., & Haegeman, J. (2025). TensorKit.jl: A Julia package for large-scale tensor computations, with a hint of category theory. *arXiv Preprint arXiv:2508.10076*. https://doi.org/10.48550/arXiv.2508.10076

Devos, L., Van Damme, M., & Haegeman, J. (2026). *MPSKit.jl* (Version v0.13.13). https://doi.org/10.5281/zenodo.10654900

Dolgov, S. V., & Savostyanov, D. V. (2014). Alternating minimal energy methods for linear systems in higher dimensions. *SIAM Journal on Scientific Computing*, *36*(5), A2248–A2271. https://doi.org/10.1137/140953289

Dolgov, S. V., & Savostyanov, D. V. (2020). Parallel cross interpolation for high-precision calculation of high-dimensional integrals. *Computer Physics Communications*, *246*, 106869. https://doi.org/10.1016/j.cpc.2019.106869

Fishman, M., White, S. R., & Stoudenmire, E. M. (2022). The ITensor software library for tensor network calculations. *SciPost Physics Codebases*, 4. https://doi.org/10.21468/SciPostPhysCodeb.4

Gelß, P., & Scikit-TT contributors. (2022). *Scikit-TT: Tensor train toolbox* (Version 1.2). https://github.com/PGelss/scikit_tt

Haegeman, J. (2026). *KrylovKit* (Version v0.10.4). Zenodo. https://doi.org/10.5281/zenodo.20820534

DRAFT

Holtz, S., Rohwedder, T., & Schneider, R. (2012). The alternating linear scheme for tensor optimization in the tensor train format. *SIAM Journal on Scientific Computing*, *34*(2), A683–A713. https://doi.org/10.1137/100818893

Kazeev, V. A., & Khoromskij, B. N. (2012). Low-rank explicit QTT representation of the Laplace operator and its inverse. *SIAM Journal on Matrix Analysis and Applications*, *33*(3), 742–758. https://doi.org/10.1137/100820479

Khoromskij, B. N. (2011). $O(d \log N)$-quantics approximation of N-d tensors in high-dimensional numerical modeling. *Constructive Approximation*, *34*(2), 257–280. https://doi.org/10.1007/s00365-011-9131-1

Khoromskij, B. N. (2018). *Tensor numerical methods in scientific computing*. De Gruyter. https://doi.org/10.1515/9783110365917

Lindsey, M. (2023). Multiscale interpolative construction of quantized tensor trains. *arXiv Preprint arXiv:2311.12554*.

Lubasch, M., Moinier, P., & Jaksch, D. (2018). Multigrid renormalization. *Journal of Computational Physics*, *372*, 587–602. https://doi.org/10.1016/j.jcp.2018.06.065

Mikkelsen, M., & Kastoryano, M. (2026). A tensor-train multidimensional inverse Laplace transform. *arXiv Preprint arXiv:2606.06093*.

Moore, J. B., Stackhouse, H. P., Fulcher, B. D., & Mahmoodian, S. (2025). Using matrix-product states for time-series machine learning. *Phys. Rev. Res.*, *7*, 043010. https://doi.org/10.1103/61h8-8qr5

Núñez Fernández, Y., Ritter, M. K., Jeannin, M., Li, J.-W., Kloss, T., Louvet, T., Terasaki, S., Parcollet, O., von Delft, J., Shinaoka, H., & Waintal, X. (2025). Learning tensor networks with tensor cross interpolation: New algorithms and libraries. *SciPost Physics*, *18*(3), 104. https://doi.org/10.21468/scipostphys.18.3.104

Oseledets, I. (2011). Tensor-train decomposition. *SIAM Journal on Scientific Computing*, *33*(5), 2295–2317. https://doi.org/10.1137/090752286

Oseledets, I., & ttpy contributors. (2017). *ttpy: Tensor train toolbox in Python* (Version 1.2.0). https://github.com/oseledets/ttpy

Oseledets, I., & TT-Toolbox contributors. (2014). *TT-Toolbox: Tensor train toolbox in MATLAB* (Version 2.3). https://github.com/oseledets/TT-toolbox

Oseledets, I., & Tyrtyshnikov, E. (2010). TT-cross approximation for multidimensional arrays. *Linear Algebra and Its Applications*, *432*(1), 70–88. https://doi.org/10.1016/j.laa.2009.07.024

White, F., Abbott, M., Zgubic, M., Revels, J., Robinson, N., Arslan, A., Müller-Widmann, D., Schaub, S. D., Ma, Y., Rackauckas, C., Will, Axen, S., Vertechi, P., Fischer, K., st–, Horikawa, Y., Cottier, B., abhro, Ranocha, H., … Haessig, P. (2026). *JuliaDiff/ChainRulesCore.jl: v1.26.1* (Version v1.26.1). Zenodo. https://doi.org/10.5281/zenodo.19386492

DRAFT